Web Engineering

Summer Semester 2026

Professor: Jürgen Cito

Assistants: Michael Schröder, Samuel Pilz, Jakob Hoffmann, Markus Böck

Tutors: Jakob Jeschke (Head TA), Dominik Kanjuh, Pawel Kusiak, Valentino Lazarevic, Lukas Oberhauser, Lukas Schatzer, Marin Toma, Franz Uchatzi

TU WIEN Informatics

Web Engineering

Mandatory (Pflichtmodul), 4th Semester:
033 526 **Wirtschaftsinformatik**

Elective (Wahlmodul):
033 521 **Informatik**

Course Structure - Theoretical Part
Flipped Classroom

Students watch lectures online and do pre-reading before attending live lectures.

Live lectures will go over “katas” (small examples to deepen understanding)
and answer student questions to theoretical aspects

04.03. Kick-off / HTTP

11.03. HTML/CSS

25.03. JavaScript

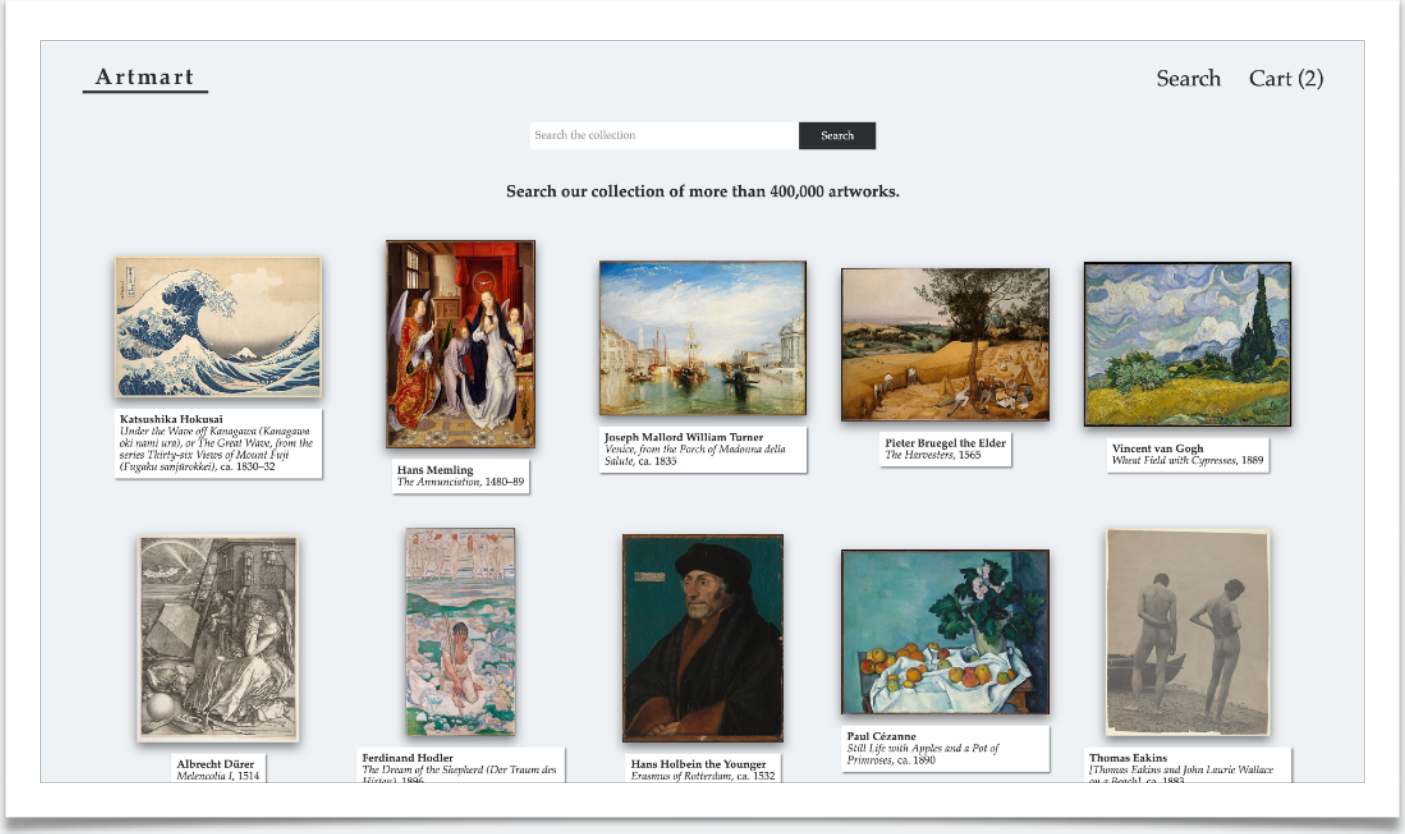
29.04. Backend Abstractions + Sockets (Node.js)

27.05. Frontend Abstractions (Vue.js)

11.06. Guest Lecture (TBD)

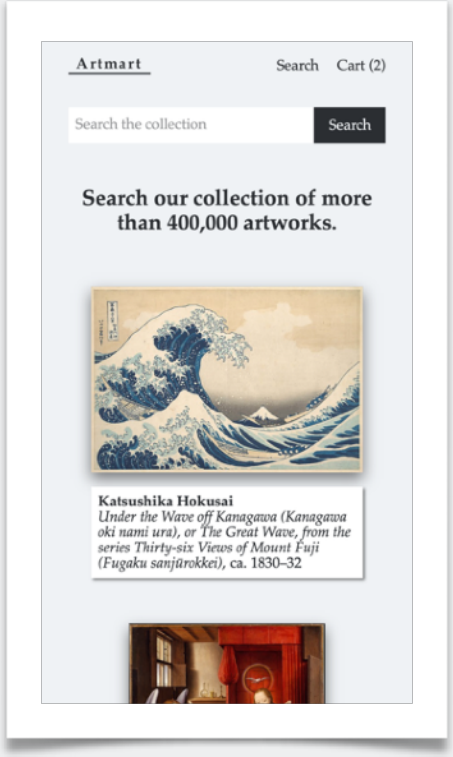
Project “Artmart”

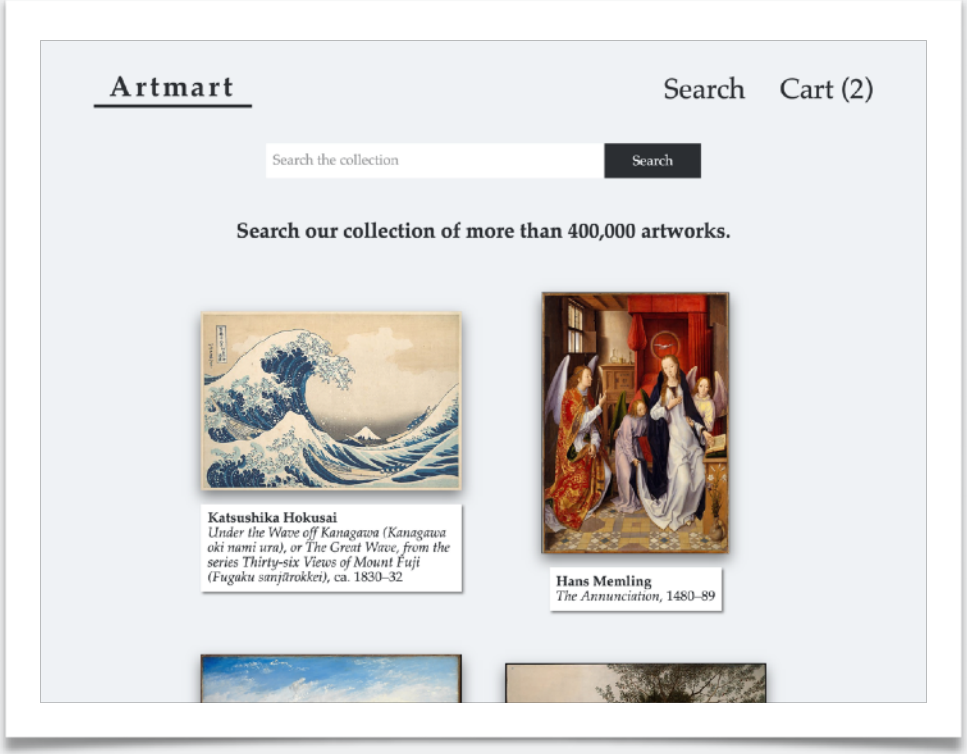
You are tasked with building the website for Artmart, a web shop for fine art prints.



Project “Artmart”

You are tasked with building the website for Artmart, a web shop for fine art prints.





Assignment 1:

Static HTML/CSS

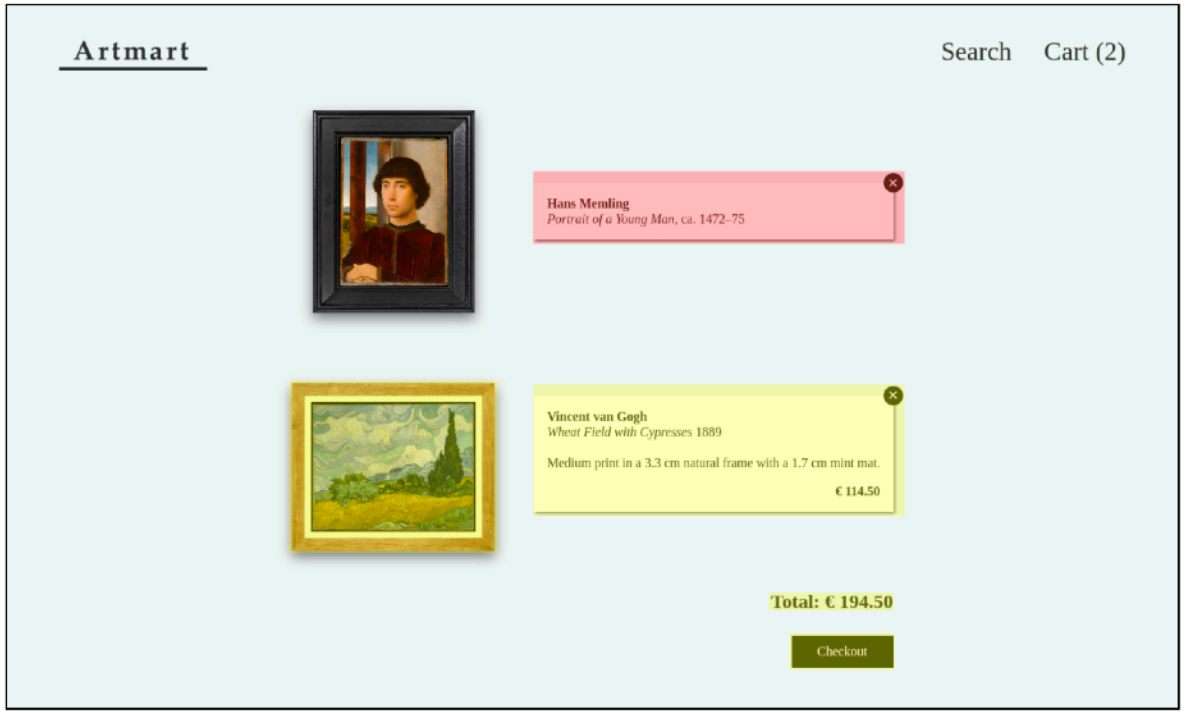
- HTML5
- WAI
- CSS3
- Responsive Design

Automated Grading A1 — Component Differencing

✘ 302 — Looks correct at 1440px width **−1**

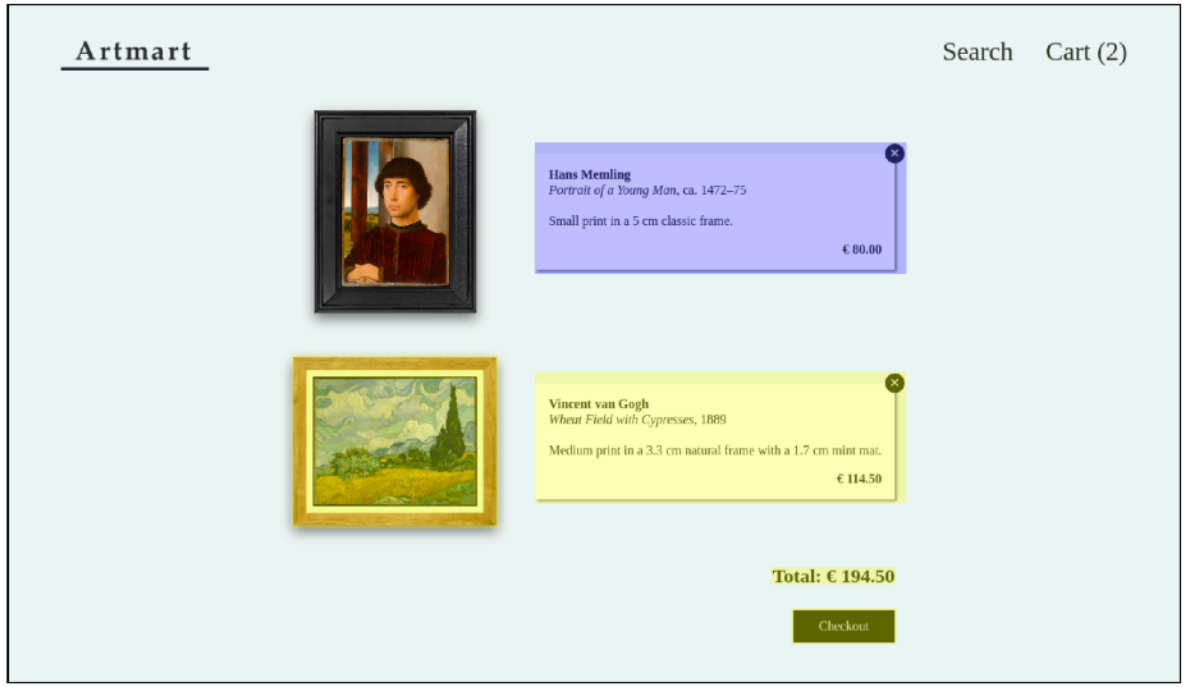
The page does not look correct: 4 components are in the wrong place, there is 1 unexpected component and 1 expected component is missing.

Your Page



- [ ] matching
- [x] moved
- [x] unexpected
- [ ] expected components

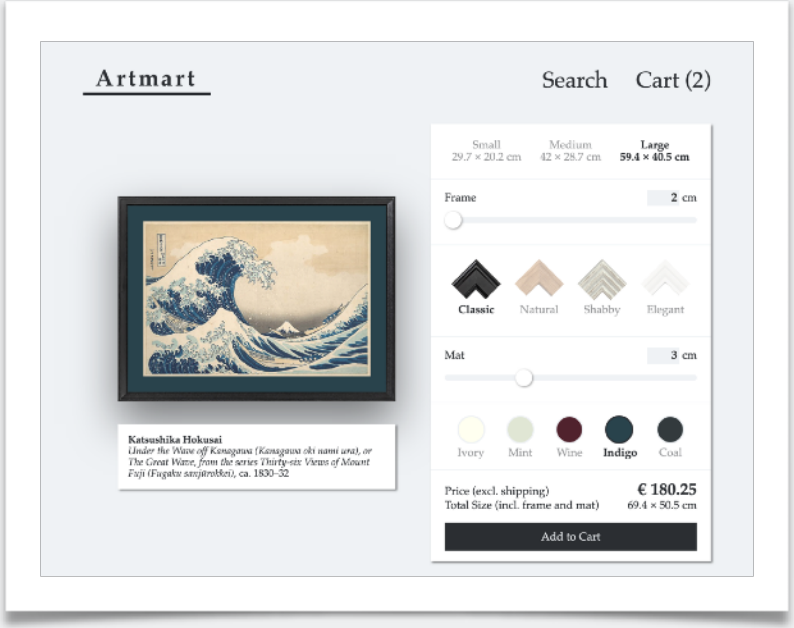
Expected

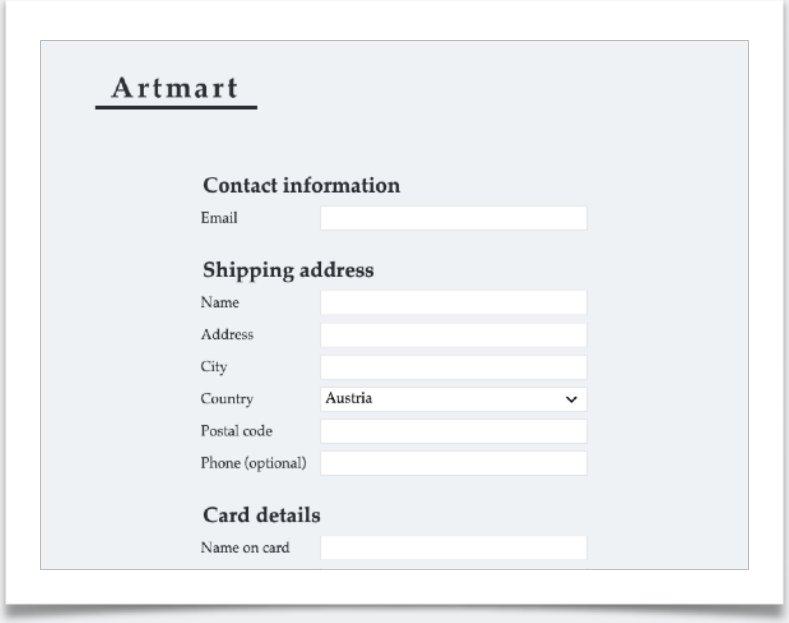


- [ ] matching
- [x] moved
- [x] missing
- [ ] your components

Project “Artmart”

You are tasked with building the website for Artmart, a web shop for fine art prints.





Assignment 2:
Web Interactivity

- JavaScript/DOM
- Service Interaction (Metropolitan Museum)

Assignment 3:
Backend

- Node.js
- REST API
- Server Sockets

Assignment 4:
Frontend Abstraction

- Vue.js
- Web Sockets

Automated Grading A2-A4 — Integration Testing

✘ 209 — Show number of items in cart

☑ start intercepting HTTP requests less info

- GET requests to relevant parts of the Met API will return random responses.
- GET requests to `images.example.com` will return a test image.
- GET requests to `localhost` will continue unaltered.
- All other requests will be aborted.

☑ put two items in the cart less info

```
localStorage.setItem('cart', JSON.stringify([
  {
    "objectID": 747867,
    "printSize": "M",
    "frameStyle": "classic",
    "frameWidth": 29,
    "matColor": "mauve",
    "matWidth": 9
  },
  {
    "objectID": 52391,
    "printSize": "L",
    "frameStyle": "elegant",
    "frameWidth": 23,
    "matColor": "mint",
    "matWidth": 8
  }
]))
```

☑ navigate to `http://localhost:4444/framing.html?objectID=99925`

☒ expect `innerText` of `nav > a[href="cart.html"]` to be `Cart (2)`

```
Expected innerText: "Cart (2)"

Actual innerText: "Cart"
```

✘ 109 — Cache Met objects

☑ start intercepting HTTP requests less info

- GET requests to relevant parts of the Met API will return random responses.
- GET requests to `images.example.com` will return a test image.
- GET requests to `localhost` will continue unaltered.
- All other requests will be aborted.

☑ navigate to `http://localhost:4444/search.html?q=vejturvo`

☒ expect Met API requests for objects 878746, 854085, 200039

```
Expected requests:
  https://collectionapi.metmuseum.org/public/collection/v1/objects/878746
  https://collectionapi.metmuseum.org/public/collection/v1/objects/854085
  https://collectionapi.metmuseum.org/public/collection/v1/objects/200039

Actual requests:
  https://collectionapi.metmuseum.org/public/collection/v1/objects/39799
  https://collectionapi.metmuseum.org/public/collection/v1/objects/459055
  https://collectionapi.metmuseum.org/public/collection/v1/objects/437853
  https://collectionapi.metmuseum.org/public/collection/v1/objects/435809
  https://collectionapi.metmuseum.org/public/collection/v1/objects/436535
  https://collectionapi.metmuseum.org/public/collection/v1/objects/360018
  https://collectionapi.metmuseum.org/public/collection/v1/objects/634108
  https://collectionapi.metmuseum.org/public/collection/v1/objects/459080
  https://collectionapi.metmuseum.org/public/collection/v1/objects/435882
  https://collectionapi.metmuseum.org/public/collection/v1/objects/271890
  https://collectionapi.metmuseum.org/public/collection/v1/objects/459054

(The order of requests does not matter.)
```

AI Policy

- **AI tools (ChatGPT, Codex, Copilot, etc.) are <u>allowed</u>**, however:
  - The goal is *learning*, not outsourcing your work.
  - AI should support your understanding, not replace it.

- **Required: AI Usage Appendix (PDF)** — for every assignment (even if no AI used)
  - AI transcript (prompts + responses)
  - link if available (e.g., ChatGPT shared link) or paste transcript (screenshots)
  - ~200-word reflection written without AI
  - *If no AI used: Statement that no AI was used*
  - What constitutes as AI use: *Use of any tool that generates code based on instructions*

- **Grading: holistic review** of assignment submission + AI transcript + reflection
  - Example: AI generates all HTML for an “HTML syntax” task → likely not acceptable
  - Iterative use (test, debug, revise using course techniques) → acceptable

Inspired by: https://designftw.mit.edu/#ai-policy

Bonus Study: AI Survey (8 points)

- **Help us study how AI affects learning in Web Engineering**
- **Participation:** short surveys (10-15 minutes) after each assignment + 1 entry and 1 exit survey
- **Topics include:**
  - AI usage during assignments
  - learning experience & difficulty
  - confidence in software engineering and AI tools
  - ownership and understanding of your code

- **Reward: +8 bonus points** for completing the survey series
  - Participation is voluntary
  - Responses are anonymized and used for research

Inspired by: https://designftw.mit.edu/#ai-policy